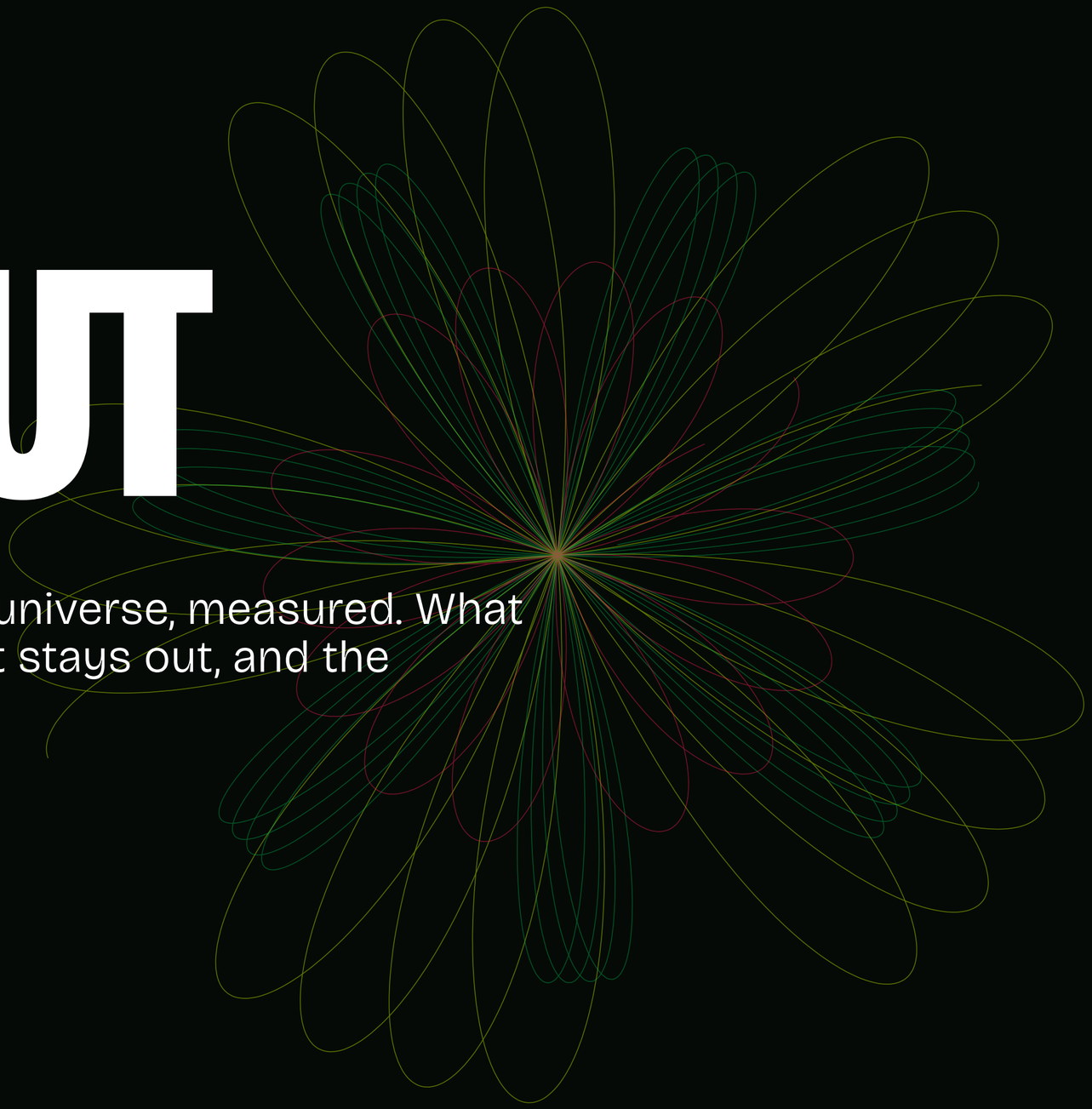
LOADOUT



Claude Code and the MCP universe, measured. What goes in a new project, what stays out, and the numbers behind each call.

TWENTY-ONE SERVERS, BOOTED IN A SANDBOX, EVERY TOOL SCHEMA COUNTED

102,616

tokens if every schema loads at startup.
283 tools.

1,415

tokens when Claude Code loads names only.
Same 283 tools.

That 72× gap is the whole argument for deferred tool loading, and Claude Code turned it on by default. Most advice about MCP context cost was written before that happened. This deck is what is true after.

THE PROBLEM WITH EVERY REVIEW YOU HAVE READ

41,384 servers in the official registry. Nobody has run them.

A RUN

We booted it, counted it, or watched it happen in a live Claude Code 2.1.296 session. Twenty-two servers, one CLI, one Cowork session with 570 deferred tools.

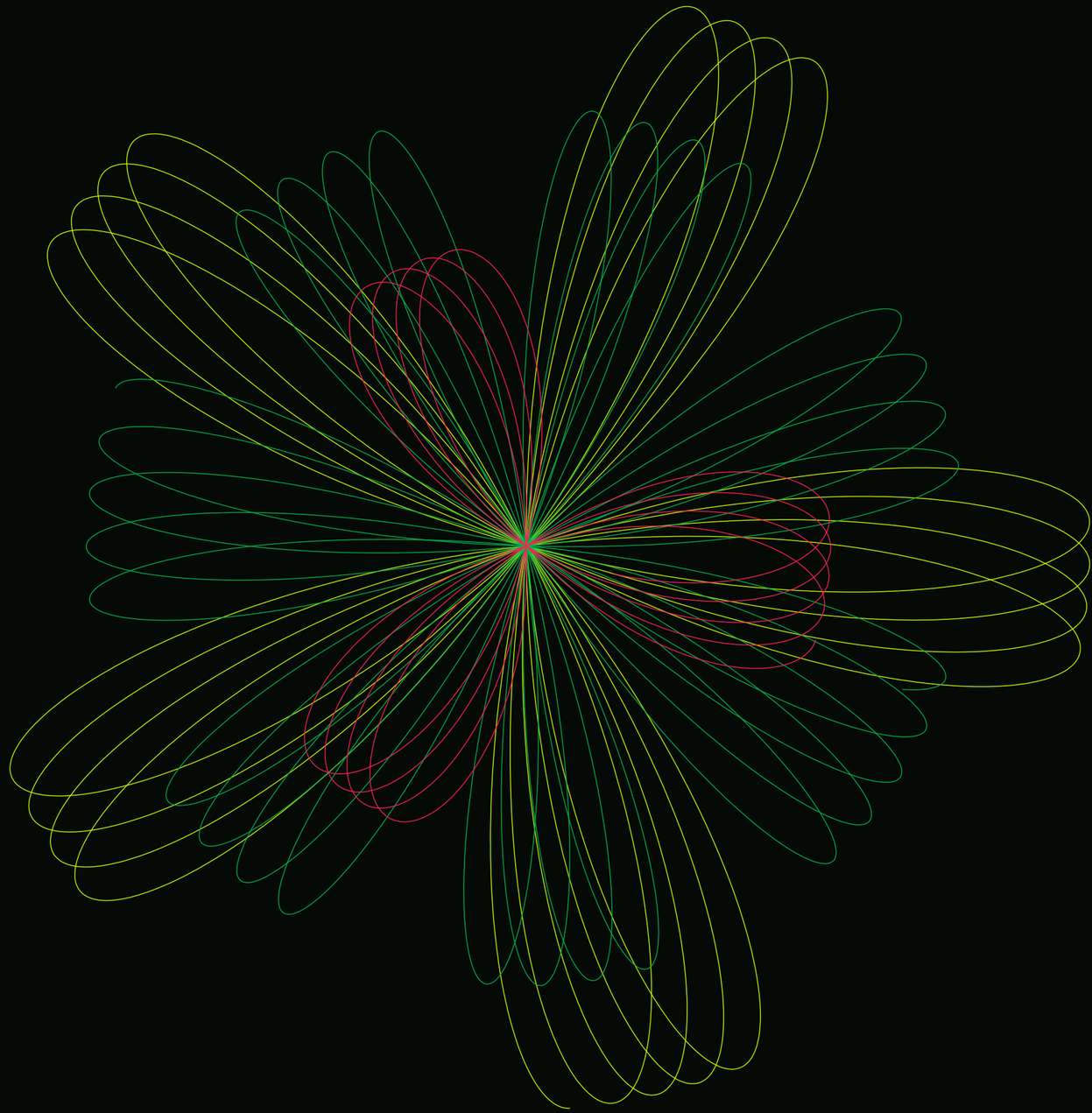
B VERIFIED

Primary documentation plus an independent signal: release page, repo activity, a published measurement. Anthropic, Palantir, MCP spec, arXiv.

C LISTED

Catalog entry only. Named so you know it exists. Never recommended on that basis.

Every claim in this deck carries its tier. Where we could not verify, we say so instead of rounding up. Fifteen UNVERIFIED items are listed in the source files and kept out of the rules.



A SETUP IS SIX JOBS. TOOLS ARE JUST WHAT FILLS A SLOT.

Always-on instruction	CLAUDE.md, rules without paths, auto memory. Loads every request.	`cost: every turn`
On-demand procedure	Skills. Name and description always, body only when invoked.	`cost: 30–50 tokens each at idle`
Guarantee	Hooks. 33 events. Deterministic. The only layer that enforces.	`cost: zero unless output returns`
Isolation	Subagents, worktrees, forks. Fresh window, summary back.	`cost: own window, 1–2K back`
External systems	CLI first. Code-mode MCP second. Remote OAuth MCP third. Local stdio last.	`cost: see the ladder`
Verification	A test the model cannot talk its way past. Stop hook, evaluator subagent, /goal.	`cost: the cheapest token you buy`

Decide the slot before you pick the tool. Most redundant installs are two tools fighting for one slot.

−29%

median runtime with an AGENTS.md present. 124 PRs, 10 repos. Output tokens −17%.

0pp

change in pass rate on SWE-bench and fresh issues. A 288-run Claude Code ablation agreed, p = 1.00.

44%

of the time a model-triggered skill fired at all in Vercel's eval. Explicit trigger words took it to 95%.

Instruction files buy efficiency and behaviour, not correctness. Repository overviews, the thing `/init` writes, did not help. Non-standard rules did. Anthropic's own April postmortem: a terse-output instruction in the system prompt cost 3% on coding evals.

Keep CLAUDE.md under 200 lines. Put only what the model cannot infer. Edits mid-session do nothing until `/clear`.

CLI FIRST.

"CLI tools are the most context-efficient way to interact with external services." Anthropic, Claude Code best practices.

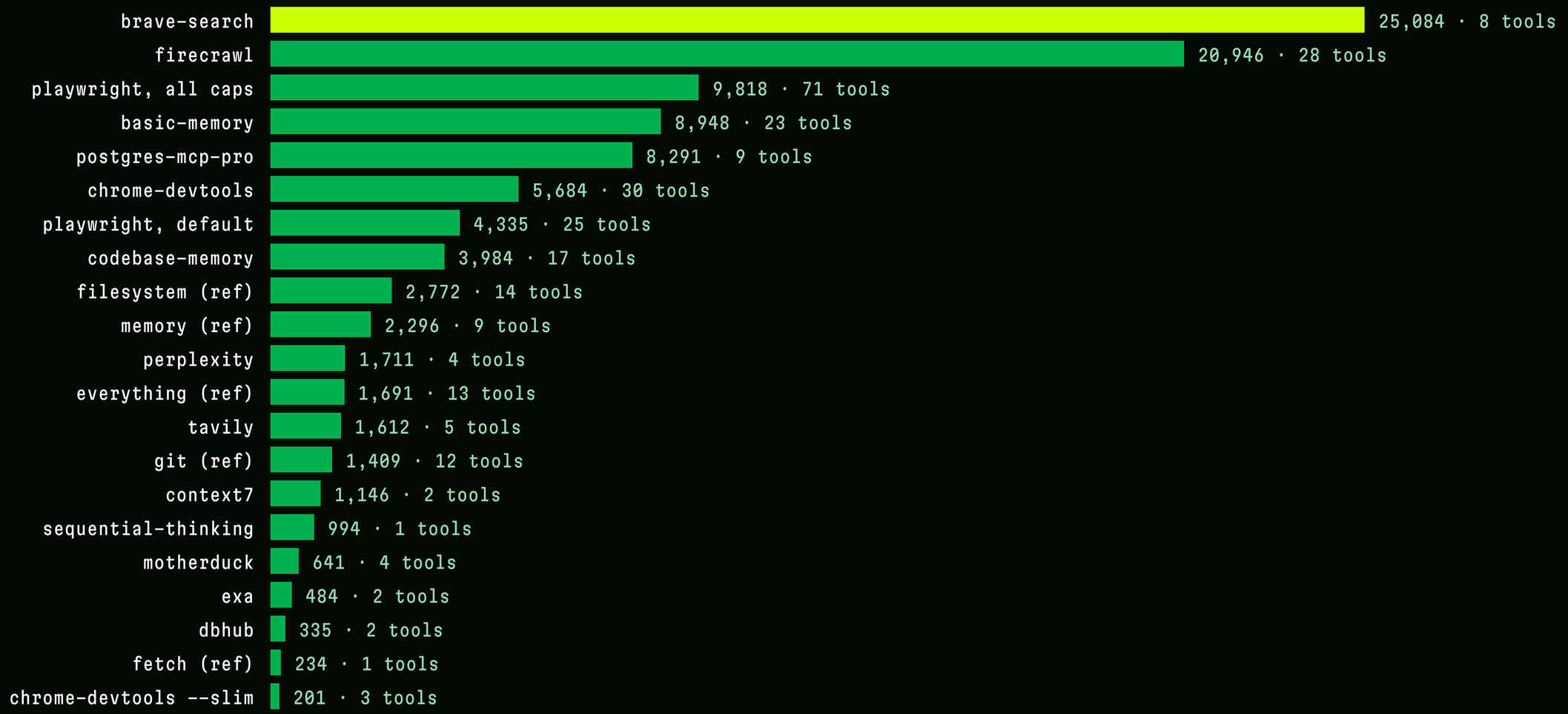
1. **CLI in Bash.** gh, vercel, wrangler, psql, git, npx @osdk/cli. No per-tool listing at all. Output lands wherever you pipe it.

2. **Code-mode or meta-tool MCP.** Cloudflare's three-tool server runs JavaScript against its API spec: 244,000 tokens of naive tools becomes 1,100. Atlassian ships discover plus execute. Palantir MCP starts with one tool, search_tools.

3. **Remote HTTP MCP with OAuth.** Where the CLI is weak or scoped auth matters: Figma, Notion, Linear, Supabase read-only, Neon, Vercel. Stateless HTTP only; the 2026-07-28 spec removed sessions.

4. **Local stdio MCP.** Only when nothing above exists. Reference servers are not hardened: the filesystem and git servers each shipped CVEs. Never install a filesystem MCP inside Claude Code; it already has Read, Edit, Grep and Bash.

SCHEMA TOKENS PER SERVER, MEASURED IN THE SANDBOX, CL100K PROXY [A]



TOOL COUNT PREDICTS NOTHING

8

tools. Brave Search.

25,084

tokens of schema.

21,462

of them belong to one tool, `brave_place_search`, whose input schema carries a giant enum.

Playwright with every capability switched on is 71 tools and 9,818 tokens. Postgres MCP Pro's nine tools cost three times Chrome DevTools' thirty. Schema shape, enums, nesting, description length, decides the bill. Count tokens, not tools. `/context all` shows you both.

PROFILES ARE THE BIGGEST LEVER A SERVER AUTHOR CAN HAND YOU [A]

chrome-devtools-mcp, full

5,684

30 tools

chrome-devtools-mcp --slim

201

3 tools. 28× cheaper.

@playwright/mcp --caps all

9,818

71 tools

@playwright/mcp default

4,335

25 tools

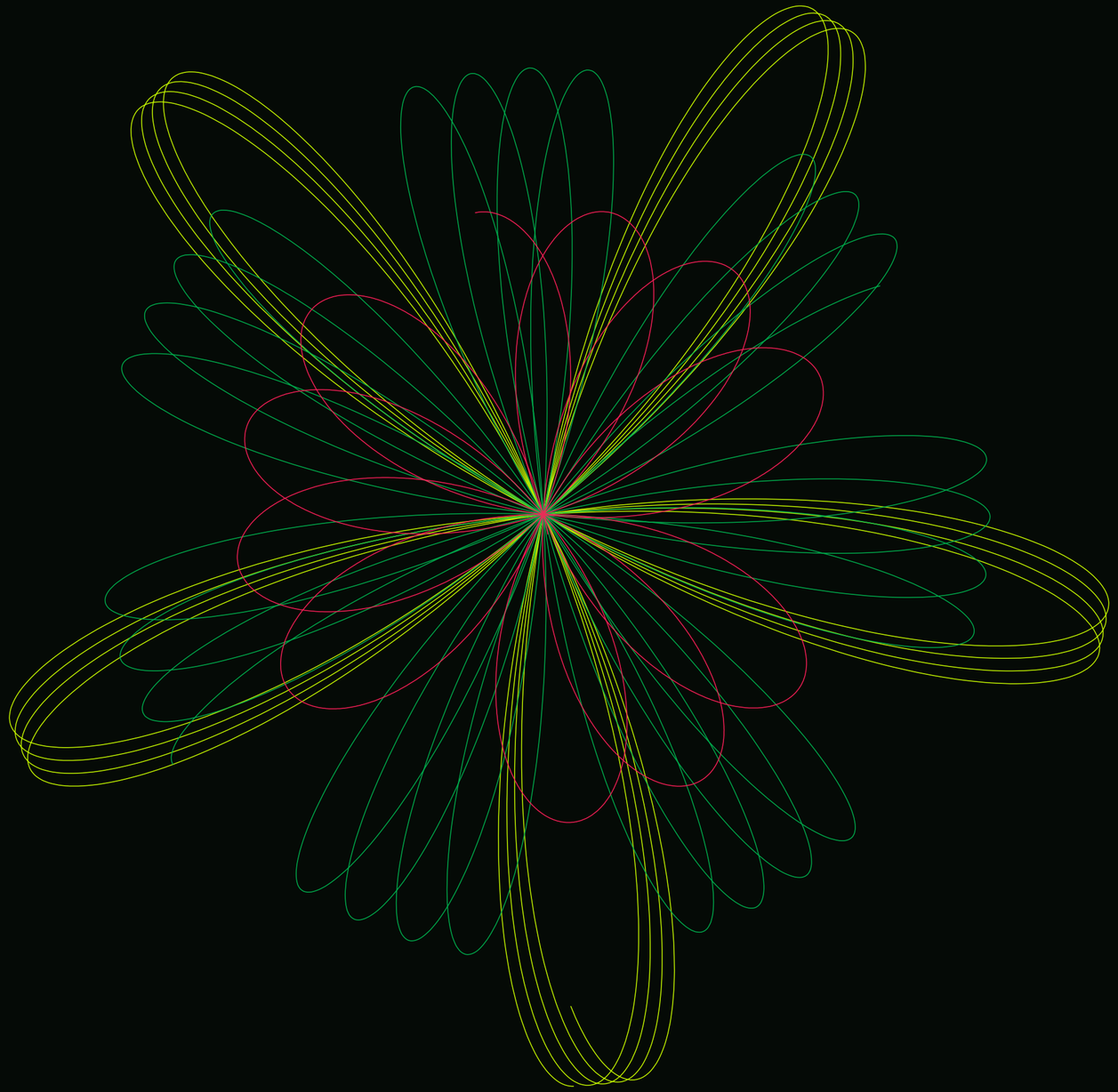
Start slim. Escalate when a task needs the performance trace or the network panel, then drop back.
One browser server, not three.

THREE COSTS DEFERRED LOADING DOES NOT TOUCH

Server instructions	Load at connect regardless. Firecrawl ships 3,254 characters of them. Context7 632. Perplexity 674. Measured. [A]
Tool results	A screenshot through MCP is 5,000 to 8,000 tokens. A file path is about 50. Claude Code truncates results at 25,000 tokens, which is a backstop, not a plan. Write big outputs to disk and return the path. [B]
Selection accuracy	Past roughly seven tool domains, models stop choosing well. GPT-4o fell to 2% on one task; retrieving tool descriptions instead of listing them lifted selection from 13.6% to 43.1%. Older models, but the mechanism is tool search. [B]

And one cost that is new: Postgres MCP Pro would not start on a fresh machine. It never pinned the MCP SDK below 2.x and the SDK renamed FastMCP. Any unpinned Python server written in 2025 is now a coin flip. [A]

Repo	Built-ins. Read, Edit, Grep, Bash.	Never a filesystem MCP.
GitHub	gh CLI.	GitHub MCP, 86 tools, only for scoped read-only subagents.
Browser	Chrome DevTools MCP --slim, escalate to full.	Playwright default 25 tools. Claude in Chrome for logged-in sessions. Pick one.
Docs	WebFetch on llms.txt. Then Context7, two tools.	DeepWiki for other people's repos.
Search	Built-in WebSearch. Exa if you want a vendor, two tools, 484 tokens.	Firecrawl for crawl only. Brave only deferred.
Database	psql or vendor CLI. dbhub if you want MCP, two tools.	Supabase or Neon remote, read_only=true.
Deploy	vercel and wrangler CLIs. Cloudflare Code Mode.	Vercel MCP exposes about 330 tools. Deferred or not at all.
Memory	CLAUDE.md plus auto memory.	Basic Memory if you want inspectable notes. No reasoning helpers.
Foundry	Palantir MCP, project scope, --tool-search. Ontology MCP per app.	@osdk/cli for site deploys. SuperRepo when your enrollment has it.



AI FDE has no API.

Palantir's in-platform builder agent went GA on 12 March 2026. It is a chat window. No MCP tool, no CLI, no endpoint. The one programmatic hook is a browser pre-fill URL, and tool approvals still happen in the browser.

So "my local Claude Code drives AI FDE agent to agent" is not a thing you can build this quarter. What Palantir built instead is better for a solo developer, because it has fewer moving parts.

THE THREE PATHS THAT EXIST

Palantir MCP

GA 2026-06-25 · about 90 tools · stdio

Claude Code does the build work itself from your IDE: object, link and action types on a branch, global proposals, Python transform repos, dataset builds, OSDK versions, Developer Console wiring. It cannot write ontology data and there is no merge tool. A human merges in the UI, by design.

Ontology MCP

GA 2026-06-25 · per Developer Console app · HTTP, OAuth

The consumer side. Object types as a SQL tool, every action type as its own tool, query functions as tools. The only MCP route that mutates data and the only one that can trigger in-platform logic.

Pro-code agent

Claude Agent SDK template · git tag to publish

The real agent-to-agent substrate. A Foundry-hosted agent pre-wired to OSDK, Ontology MCP and Palantir MCP with scoped permissions and no secrets. Expose it through an action. Claude Code fires the action through Ontology MCP and reads results back from objects.

```
claude mcp add palantir-mcp --scope project -e FOUNDRY_TOKEN='${FOUNDRY_TOKEN}' -- npx -y palantir-mcp --foundry-api-url
https://$FOUNDRY_HOST --tool-search
```

FREE DEVELOPER TIER, THE WALLS YOU WILL HIT [B]

60
action types, hard cap

~5
users

0
client-credentials OAuth grants. Auth-code only, so no headless service user for Ontology MCP.

GA
apps only. SuperRepo, the Foundry CLI and Pilot are beta and need a support ticket.

The model inside AIP is not the weak link: Opus 5.5, Opus 5 and Sonnet 5 are all served. The limit is tokens per minute. Delete the pre-installed example package to recover ontology quota. Numeric compute and storage caps are not published; they live in Control Panel under Your Plan.

FOCUSED
BEATS FULL.

76%

Opus 4.6 on 8-needle retrieval at one million tokens. Sonnet 4.5: 18.5%. The best public curve there is.

18/18

models did better on a 300-token focused input than on the 113,000-token full input, thinking on. Chroma, 194,480 calls.

−39%

multi-turn versus single-turn across 200,000 simulated conversations. Mostly unreliability, not lost ability.

No cliff. Monotonic decay that starts early for anything that is not a keyword match. One million tokens is headroom for a large read, not permission for a day-long session. `/clear` between unrelated tasks. `/rewind` is cheaper than `/compact`. Front-load the whole spec in one prompt.

One /model switch on a 150K session costs about fifteen to twenty normal turns.

Warm turn: new tokens at the write rate, plus the prefix at a tenth of the input price. Cold turn: the whole prefix at 1.25× or 2×.

Breakers: model switch, effort change on older models, connecting an MCP server with tool search off, enabling a plugin that ships MCP, upgrading the CLI, a skill with a model: line.

Harmless: editing files, editing CLAUDE.md, permission modes, invoking skills, spawning subagents.

Pick model and effort before the first message. Connect servers before the first message. Subscriptions get a one-hour cache inside plan usage; API keys and subagents get five minutes. Anthropic's own team pages on cache-hit rate.

15×

tokens for Anthropic's multi-agent research system against a chat. It bought +90% on research. Anthropic's own verdict on coding: fewer parallelizable parts.

44%

of multi-agent failures trace to specification and role design. 1,642 traces, seven frameworks. Changing topology was not significant. Changing the prompt was.

117

jobs accepted out of 2.4 million requests when 80 agents shared a queue with no locks. Same study: the models that merged work owned their files.

Parallelism pays for read-heavy, independent work with a cheap merge: research, review, hunting. Sequential coding wants one agent. When you do fan out: disjoint file ownership, file-based task locks, an objective verifier, results written to disk and referenced, never pasted.

THE PERMISSION NUMBERS NOBODY PUTS ON A SLIDE B

93%
of permission prompts get approved. That is fatigue, not judgement.

84%
fewer prompts with the OS sandbox on. It is off by default.

17%
of overeager actions get past auto mode. Anthropic calls it the honest number.

23
pull requests in 74 minutes added a server that swapped its tool list after the third call to hunt credentials. August 2026.

Mods run in-process with your permissions and can override your own hooks. CLAUDE.md and memory directories reload every session, which makes them a persistence vector. Install from the vendor's URL or package, never through a directory's proxy. Two Anthropic reference servers shipped CVEs; reference is not hardened.

1. CLAUDE.md under 200 lines: build and test commands, the host names, the rules the model cannot infer. One `.claude/rules` file per sub-system with `paths:`.

2. Install the CLIs. gh, vercel, wrangler, the vendor database client. Confirm each works in Bash before any server is added.

3. Commit a `.mcp.json` with only what the ladder justifies, slim profiles on, tool search on. Then run `/context all` and read the bill.

4. Three skills with explicit trigger words: the deploy path, the review path with `context: fork`, the thing you do weekly and keep re-explaining.

5. Hooks for the guarantees: format on edit, block pushes to main, a Stop hook that runs the suite and greps the output down to the failures.

6. Turn the sandbox on. Pick model and effort. Then, and only then, send the first message.

Every harness component is an assumption about what the model cannot do.

Anthropic's context resets, built for Sonnet 4.5, were dead weight on Opus 4.5 and removed. Their sprint contracts lasted one generation. The default effort level changed twice this year and one of the changes was a mistake they reversed.

So the setup needs a ritual, not a config: on each model release, remove one component at a time and re-run the three skills and the Stop hook. Keep what still earns its tokens. The source files behind this deck list fifteen things we could not verify, so you know where the edges are.

METHOD AND SOURCES

Twenty-two MCP servers booted over stdio with the official Python client, 2026-10-10, protocol 2025-11-25 on every one. Tool schemas dumped and counted with tiktoken cl100k as a proxy; Claude's tokenizer runs roughly 10 to 20% higher on JSON, so read the figures as relative. Claude Code 2.1.296 flags verified against the installed binary. Connector counts observed in a live Cowork session.

Primary sources: code.claude.com docs and changelog through 2.1.296; anthropic.com engineering and research, 2025 to 2026; palantir.com/docs Foundry, AI FDE, Palantir MCP, Ontology MCP, SuperRepo, announcements March to October 2026; modelcontextprotocol.io specification 2026-07-28; arXiv 2602.11988, 2601.20404, 2607.27250, 2503.13657, 2505.06120, 2605.12922; Chroma Context Rot; METR incident investigation; Cloud Security Alliance research notes.

UNVERIFIED AND EXCLUDED FROM RULES: changelog 2.1.279 TO 2.1.286 · PLUGIN MEASURE COMMAND · OMCP TRANSPORT · DEV-TIER COMPUTE QUOTAS · TOOL-COUNT ACCURACY ON 4.5+ MODELS · SEQUENTIAL-THINKING BENEFIT · PALANTIR MCP SCHEMA SIZE · 8 MORE IN THE LEDGER

